QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

By: hackathon4dummies

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - Team **Gold Digger**
Team Repo URL	https://github.com/lydianzr/Gold-Digger-AI-Assistant
Project Board URL	*Trello_Project Board*

Objective:

The primary objective of Gold Digger AI is to develop a reliable AI-powered system that analyzes gold market data and user-specific financial goals to provide actionable, explainable recommendations that enhance user confidence in decision-making by enabling more informed choices and optimizing their gold investment decisions under scalable and efficient system performance.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns the testing with specific user requirements via the **Requirement Traceability Matrix.** So, it ensures every test has been conducted to prevent bugs/failure in products for those specific requirements and unplanned feature creep.

1.1 In-Scope Core Features

- **Gold Price Analysis**: Display current gold prices (RM/gram), historical trends, and basic market insights to support decision-making.
- **'Buy' Decision Engine**: Allow users to input budget, investment goal, and preferences to receive AI-generated recommendations on whether to buy gold, including suggested strategies and explanations.
- **AI Recommendation (Z.AI GLM core)**: Generate context-aware, actionable recommendations with clear explanations using the Z.AI GLM as the central reasoning engine.

1.2 Out-of-Scope

- **Real-Time Trading/ Transactions**: The system will not support actual buying of gold or integration with financial institutions for transactions.
- **User Financial Account Integration**: No direct integration with bank accounts, e-wallets, or real financial data.
- **User Profile Managemen**t: Editing personal profiles, storing personal financial history, or managing user accounts is excluded.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

Quality assurance risks are anticipatory. Therefore, technical risks associated with the application requirements and system architecture are identified. The risks are then evaluated using a 5×5 risk assessment matrix, where the risk score is calculated by multiplying likelihood and severity (Risk Score = Likelihood × Severity).

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
GLM service unavailable or API failure	3	5	15 (High)	Implement fallback response (e.g., “Unable to generate recommendation”); retry mechanism; timeout handling	Simulate API failure and observe system response and error handling
Inaccurate or outdated gold price data	4	4	16 (High)	Use reliable data sources; implement periodic updates; display timestamp of last update	Check accuracy against trusted sources, while ensuring the system handles API failure using fallback data.
Incorrect AI recommendation due to poor or inconsistent input data	3	5	15 (High)	Validate and sanitize user inputs; provide default assumptions when data is missing; include an explanation layer to justify decisions	Test with incomplete, extreme, and invalid inputs; verify consistency of AI outputs
Misinterpretation of user intent (e.g.: wrong goal classification)	3	4	12 (High)	Use structured input fields instead of free text; guide users with predefined options	Test different combinations of user inputs and verify recommendation relevance

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

The testing environment for the application is defined, where testing is conducted, how test data is managed, and the rules and thresholds used to control each testing phase.

- **Unit Test**
  - Scope: Buy Decision Calculations, Input Validations
  - Execution: Unit tests are written using frameworks such as PyTest and are executed during development (locally) and automatically in CI pipeline upon code push
  - Isolation: Testing focused only on internal logic, external dependencies, such as Gold Price APIs and Z.AI GLM API responses, are mocked
  - Pass Condition: All expected outputs match predefined results

- **Integration Test**
  - Scope: Interaction between Frontend and Backend API
  - Execution: This test is performed after proper completion of Unit Testing, after merging feature branches
  - Workflow: To ensure the API calls are working, so real calls are tested without mocking.
  - Pass Condition: API responses are accurate and properly rendered on UI, and no data mismatch or communication failure

- **Test Environment (CI/CD Practice):**
  - Local Testing: Features tested manually on localhost
  - Staging/CI: GitHub Action triggers tests on every code push
  - CI/CD Automated Pipeline: GitHub Actions does basic tests for branch specific level and then PyTest for PR (Pull requests) to the main branch

- **Regression Testing & Pass/Fail Rules:**
  - Execution Phase: This testing tests across all in-scope features each time a new branch is merged to the main branch
  - Pass/Fail Condition: Test case pass is only considered when actual outcome perfectly aligns with the expected one. If mismatch/threshold is not achieved, the test status is considered a failure
  - Continuation Rule: While gradually doing testing, the critical modules must pass first and meet the threshold to allow continuation for further testing

- **Test Data Strategy:**
  - Manual: Tested different user scenarios (e.g., different budgets)
  - Automated: Use of predefined datasets for gold price trends

- **Passing Rate Threshold:**
  - A minimum of 85% of all test cases must pass for the system to be considered acceptable. However, critical tests including AI recommendation generation and API communication with the Z.AI GLM must achieve a 100% pass rate with no exceptions.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Any failure in these critical areas will result in deployment being blocked until the issues are resolved.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. Thus, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	Zero Build Error	0 Error	Passed
Unit Tests	100% Passing Rate	100%	Passed
Code Quality	Zero Linting Error	100%	Passed
Test Coverage	Minimum 85%	75%	Failed

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	Minimum 90%	91%	Passed
AI Recommendation Accuracy	≥ 85% correct vs expected scenarios	85%	Passed
Explanation Clarity (GLM Output)	Clear reasoning in all outputs (100%)	100%	Passed
Critical Bugs	Zero P0/01 Bugs	0	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

API Performance	Response Time < 800ms	1100ms	Failed
Security	API keys are not exposed	Clean	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Test Case Specifications (Drafts)

This section is for you to draft your test cases. The test cases MUST include at least:

- ***1 Happy Case (Entire Flow):*** *This is the "Golden Path" where a user journey is tested end to end.*
- ***1 Specific Case (Negative/Edge Test):*** *This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.*
- **2 Non-Functional (NFR) Tests (Performance/Load)*****:*** *This test consists of non-feature-based tests but architectural tests.*

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	**Happy Case (Entire Flow): Buy Decision Engine + GLM Recommendation**	Verify whether a user can input budget and preferences, receive AI recommendations, and view explanations correctly. Ensures full system flow works.	1. Open web app 2. Enter budget (RM5000) 3. Select goal (Long-term investment) 4. Choose Risk Tolerance (Low) 5. Click "Analyze Now" 6. System processes request via GLM 7. Display recommendation and explanation	System generates recommendation (e.g., "Buy") with explanation within an acceptable response time (< 800ms). Output includes reasoning and estimated value.	To Be Tested
TC-02	**Specific Case (Negative): Invalid Input Handling**	Verify system handles invalid input (e.g., negative budget or empty fields) without crashing and provides proper error feedback.	1. Open Start Analysis 2. Enter invalid budget (e.g., -1000 or empty) 3. Click "Analyze Now"	System blocks request and displays clear error message (e.g., "Enter valid budget"). No API call should be made.	To Be Tested
TC-03	**NFR (Performance): GLM API Response Time**	Verify that the GLM-based recommendation system responds within acceptable	1. Simulate 50 concurrent users sending recommendation requests	Average response time < 800ms with stable responses and no failures	To Be Tested

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
		time under concurrent usage.	2. Monitor API response time 3. Record average latency		
TC-04	**NFR (Load): System Stability Under Load**	Verify system stability when multiple users simultaneously use Analysis features.	1. Simulate 100 concurrent users interacting with system 2. Perform mixed operations (buy, sell, compare) 3. Monitor system performance and errors	System remains stable with no crashes, minimal latency increase, and <5% error rate	To Be Tested

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that the Z.AI GLM integration produces acceptable output and handles any inputs that are abnormal gracefully.

6.1. Prompt/Response Test Pairs

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	User: "Budget RM5000, Goal: Long-term investment, Risk Tolerance: Low"	The system must generate a clear decision indicating whether the user should buy or hold, along with a	To Be Tested	Pending

© UMHackathon 2026

Email: umhackathon@um.edu.my

		confidence score (0–100%) reflecting the strength of the recommendation, and also provide a smart buying strategy. The response must be explainable, consistent with the input, and must not contain any hallucinated or unsupported financial claims.		
AI-02	User: “Budget RM500, Goal: Short-term trading, Risk Tolerance: High”	Output must compare trade-offs, highlight risks, and not blindly agree with user preferences. Must include reasoning for recommendation override if needed.	To Be Tested	Pending
AI-03	AI-Chat- User: “Is it better to buy gold now or wait for the price to drop?”	The system must accept a user-generated, free-form question related to gold investment and produce a context-aware response that correctly interprets the user’s intent, provides a clear and reasoned explanation based	To Be Tested	Pending

© UMHackathon 2026

Email: umhackathon@um.edu.my

		on relevant gold market logic such as trends or inflation effects, and delivers a balanced recommendation (e.g., buy, hold, or wait). The response must remain conversational and easy to understand, avoid any hallucinated or unsupported financial information, and may include a confidence level or reasoning strength indicator to support the recommendation.		

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	1500 Token/800 Word Count
Input used while testing	9000 Words - 6x of the limit
Expected Behavior	System should reject input or intelligently truncate and process in segments without losing decision accuracy
Actual Behavior	To Be Tested
Status	Pending

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.3. Adversarial/Edge Prompt Test

In this section, test minimum of 1 with one prompt that is ambiguous, malformed - document how GLM does respond and how your system is going to handle it.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-04	"I have money… maybe RM?? I want gold or something good for future maybe also jewellery??"	System should request clarification or structure input into assumptions (budget unknown, goal unclear). Must not hallucinate exact values. Must provide safe fallback suggestion.	To Be Tested	Pending

6.4. Hallucinating Handling

The Gold Digger AI system mitigates hallucinations through structured prompting, constrained decision logic, and validation rules applied before displaying AI outputs. The GLM is restricted to predefined financial reasoning boundaries, such as market-based assumptions, and user-provided inputs only. Any missing or ambiguous data triggers clarification prompts or fallback logic instead of speculative outputs. Additionally, all AI-generated recommendations are cross-checked against rule-based constraints (e.g., budget feasibility and gold pricing structure) before being displayed to the user. This ensures that the system does not fabricate financial data or produce misleading investment advice.

Important Note:

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.

© UMHackathon 2026

Email: umhackathon@um.edu.my